

Keywords:

LaTeX

I. Introduction

The rendering pipeline consists of four stages, summarized in Table ??.

Our contributions are three-fold:

- A token-stream based parser supporting common LaTeX{} macro constructs.
- A full two-column layout engine with Knuth–Plass optimal line breaking.
- A math layout engine accurate to 0.1\pt for display-mode expressions.

Figure 1. Rendering pipeline overview. Input LaTeX source is tokenised, expanded, parsed into an AST, and finally typeset into CoreGraphics draw calls.

II. Related Work

A. Server-Side Compilation

Overleaf [1] and similar services compile LaTeX{} remotely. This approach is feature-complete but requires a network connection, has latency of several seconds per compile, and raises privacy concerns for sensitive documents.

B. Native Renderers

Previous native implementations exist for iOS (iTTeX, TeX Writer Pro) but achieve limited fidelity due to simplified line-breaking and absence of proper math layout.

III. System Architecture

The rendering pipeline consists of four stages, summarized in Table ??. Fig. ?? shows a high-level overview of the architecture.



Table: Rendering Pipeline Stages

A. Mathematical Typesetting

The math engine implements a subset of TeX box construction rules:

$$E=mc^2$$

For multi-line aligned equations:

$$f(x) \&= x^2 + 2x + 1 \setminus \setminus g(x) \&= \sqrt{x^2 + 1}$$

The axis height  $\sigma_{22}$  is set to  $0.258 f_s$  where  $f_s$  is the design size, matching Computer Modern metrics.

IV. Evaluation

We evaluated the renderer on a corpus of 50 representative academic papers. Mean per-page SSIM compared to Overleaf output was  $0.94 \pm 0.02$ . The primary deficit was in complex TikZ figures (excluded from corpus).

V. Conclusion

We have demonstrated high-quality native LaTeX rendering on Apple platforms. Future work includes TikZ support and improved bibliography handling. Code available at <https://github.com/example/typetex>.